UNIVERSITÉ SORBONNE PARIS NORD

Développement d'application

Théo PALUGAN
Karl DIRIL
Manase SAMUEL DIAKOLO
Lucas CHEN

BUT 2 FA Informatique

Équipe : 404 (C)

Groupe : Margaret

Année 2026 - 2027

IUT de Villetaneuse (Sorbonne université paris 13)

Développement d'application (SAE S301)

05/10/2026 - 18/10/2026

Livrable 3 : Plateforme de développement

Pour ce livrable, nous avons décidé d'expliquer dans un premier temps le choix de nos logiciels en précisant pourquoi ce logiciel est adapté à nos besoins et ce qui nous attire sur le logiciel. Dans un second temps, les précisions sur les logiciels.

1. Explication des choix de logiciel

Visual Studio Code :

Ici on a décidé de choisir Visual Studio Code comme environnement de développement (IDE) car il est gratuit, facile à utiliser et compatible avec divers langages de programmation. Il offre aussi de multiples extensions qui rendent le développement et le débogage du code plus simples. De plus, son intégration avec Git et GitHub nous offre la possibilité de collaborer en équipe, de partager notre code et de suivre les modifications effectuées sur le projet.

PostgreSQL:

Nous avons choisi d'utiliser PostgreSQL comme système de gestion de base de données (SGBD), principalement en raison des connaissances et des compétences acquises lors de notre formation, ce qui nous permet d'avoir une certaine maîtrise de cet outil. De plus, il est compatible avec Looping AGL, que nous utilisons pour modéliser notre base de données, ce qui facilite sa mise en place.

PHP :

Nous permet d'adopter une architecture structurée et claire, et elle permet de dynamiser un site web. Il permet d'adapter les pages web en temps réel, par exemple d'afficher le nombre d'utilisateurs connectés en temps réel, il permet également de faire le lien entre la base de données et le serveur. Il pourrait aussi nous permettre de créer des classes, les objets seront par exemple des utilisateurs, des films ou des séries. Enfin, il nous permettrait de gérer les sessions et les cookies de chaque utilisateur.

Apache (apache2) :

Nous servira de serveur web pour héberger toute notre application web, il permettra de faire le lien entre le système d'exploitation "linux" et le langage PHP. Il écoutera les

demandes de chaque utilisateur sur le site, par exemple une recherche, et leur renverra la bonne page avec les bonnes données. Il y a également l'aspect de la sécurité, en effet Apache permet de chiffrer des données sensibles, de mettre en place des authentifications par mot de passe et de mettre des restrictions par IP.

Looping :

Looping est un logiciel qui permet la modélisation d'une base de données, Looping est donc une AGL qui va nous permettre de générer du code SQL adapté à PostgreSQL de façon très interactive, il permet de faire des modélisation très proche de ce que nous réalisons en cours, ce qui nous permet de ne pas être perdu, mais ce qui le différencier de MySQL Workbench par exemple et le fait qu'il soit compatible avec PostgreSQL ce qui va nous permettre de gagner du temps et d'éviter de devoir modifier le code généré par d'autre AGL.

Grafana :

Grafana est un logiciel qui nous offre la possibilité de visualiser des données grâce à une base de données en envoyant des requêtes SQL pour ensuite faire des analyses des résultats obtenus, avec par exemple la réalisation de tableaux, graphiques, cartes géographiques, courbes, … Nous avons choisi Grafana et pas un autre logiciel, car Grafana est compatible avec plusieurs sources de données, ce qui peut être utile si nous devions utiliser plusieurs systèmes différents.

Le choix du JS Vanilla à la place de JQuery :

jQuery est une bibliothèque JavaScript qui permet de faciliter la manipulation des éléments DOM, des event listeners et des animations. Cela nous permettra de dynamiser l'interface utilisateur côté client et de communiquer avec les scripts PHP côté serveur. Bien que jQuery soit suggéré dans le sujet, nous faisons le choix d'utiliser du JavaScript natif moderne (ES6+). L'API fetch et les méthodes standards du DOM couvrent l'ensemble de nos besoins et nous estimons donc que JQuery ne va qu'alourdir le projet. De plus, si on regarde du côté de Bootstrap 5, le jQuery n'est plus le support principal car le JS moderne a grandement évolué, rendant jQuery inutile. Cela nous donne une plus grande flexibilité et lisibilité au niveau du code, notamment avec l'utilisation des modules. Cela nous permet donc d'alléger le projet, donc de rendre les pages plus rapides à charger.

Bootstrap :

Pour la conception de l'interface utilisateur, nous avons retenu Bootstrap (version 5) ainsi que sa bibliothèque d'icônes Bootstrap Icons, conformément à ce qui a été recommandé dans le sujet de la SAE.

Face aux tendances actuelles du développement frontend, dominé par des approches par framework tels que Tailwind CSS, le choix de Bootstrap reste quand même pertinent pour notre projet car notre application requiert un affichage assez structuré de nombreuses informations issues de la base. Bootstrap met à disposition un ensemble de composants prêts à l'emploi qui vont accélérer le développement frontend.

De plus, contrairement à Tailwind CSS qui nécessite l'installation d'outils pour compiler le CSS, Bootstrap 5 peut être intégré beaucoup plus facilement, sans alourdir la configuration de notre environnement.

Depuis sa version 5, Bootstrap s'appuie sur le JS vanilla, cela coïncide donc bien avec le reste de notre application et cela permet aussi un temps de rendu fluide pour l'utilisateur final.

2. Précisions logiciels

Configuration initial :

- Visual Studio Code version 1.142; extension PHP, SQL, Git, …
- PostgreSQL version 18.6.0;
- PHP version 8.5.11 avec PHP-FPM; extension PHP pdo_pgsql
- Apache (apache2) en mode Worker version 2.4.69; module : mod_headers, mod_ssl, mod_proxy_fcgi et mod_proxy.
- Looping version 4.1
- Grafana version 13.2.4
- JavaScript version ECMAScript 2015
- Bootstrap version 5.3.8

D'après nos recherches sur les différents logiciels dont nous pensons avoir besoin pour réaliser cette SAE, nous avons décidé de prendre les versions les plus récentes dans la plupart des logiciels, nous pensons que toutes ces versions sont compatibles entre

elles. Pour VS Code, nous avons décidé d'ajouter des extensions pour faciliter la lecture du code et la gestion des erreurs.

Pour Apache, nous pensons avoir besoin d'utiliser le mode Worker et non le mode Prefork ou Event, car le mode Worker peut traiter des requêtes simultanément avec un processus, ce qui n'est pas le cas de Prefork et Event. Lui se focalise plus sur les connexions, ce qui nous intéresse moins dans notre cas.

Nous pensons également avoir besoin de *mod_proxy* qui devrait permettre la transmission de requêtes PHP à PHP-FPM et le module *mod_proxy_fcgi* doit permettre à Apache de communiquer avec PHP-FPM. Nous avons également trouvé que les modules *mod_header* et *mod_ssl* devraient nous être utiles pour sécuriser les en-têtes HTTP et les connexions HTTPS

Pour PHP-FPM, nous pensons utiliser ce service pour améliorer les performances de notre application, c'est-a-dire gérer le trafic et les calculs lourds, par exemple faire tourner les algorithmes rapidement sans planter ni ralentir. Il permet donc de choisir un processus disponible pour exécuter du code PHP.

Nous pensons que l'extension *pdo_pgsql* est nécessaire pour ce projet car, conformément aux exigences de sécurité du projet, il demande des requêtes préparées obligatoires (PDO) pour lutter contre les injections SQL, cette extension nous permet donc de le mettre en place.

Ceci représente une estimation des logiciels dont nous aurons besoin, néanmoins, si cela s'avère inutile ou si nous avons besoin d'autres logiciels, nous modifierons ou ajouterons des éléments par la suite.